

Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning ¹
Student Name	Danusu K
Register Number	3122247001013
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	September 20, 2026

1 Objective

- To implement **Model A: Single-Layer Perceptron Learning Algorithm (PLA)** from scratch (step activation, classical weight-update rule) as a multiclass One-vs-Rest classifier.
- To implement **Model B: Multilayer Perceptron (MLP)** with hidden layers and non-linear activations, trained via backpropagation.
- To select and justify MLP hyperparameters — activation function, cost function, optimizer, learning rate, hidden-layer architecture, and batch size — through systematic staged hyperparameter tuning.
- To rigorously compare PLA and tuned MLP using Accuracy, Precision, Recall, F1-score, confusion matrices, micro/macro-average ROC curves, and training convergence curves, on the **English Handwritten Characters** dataset (62 classes: 0–9, A–Z, a–z).

2 Problem Statement

Handwritten character recognition is a classical, genuinely non-linear pattern-recognition problem: strokes forming visually similar characters (e.g., '0'/'O', 'l'/'I'/'1') cannot be separated by simple linear decision boundaries in raw pixel space. This experiment uses a small, balanced 62-class handwritten character dataset (3,410 images, 55 per class) to directly contrast a purely linear classifier (the Perceptron) against a non-linear, hidden-layer network (the MLP), quantifying exactly how much representational and optimization power is gained by moving from a single trainable linear layer to a tuned, multi-parameter non-linear architecture — while also confronting the practical challenges (dying activations, optimizer divergence, overfitting with limited per-class samples) that arise when tuning a neural network on a small dataset.

3 Theoretical Background

3.1 Perceptron Learning Algorithm (PLA)

The perceptron computes a linear decision function using a step activation:

$$\hat{y} = \begin{cases} 1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b \geq 0 \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

¹GitHub Repository: <https://github.com/Danush-k/Machine-Learning>

and updates its weights using the classical rule:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \eta(y - \hat{y}) \mathbf{x} \quad (2)$$

For a K -class problem, PLA is extended via the **One-vs-Rest (OvR)** scheme: K independent binary perceptrons $\{\mathbf{w}_c\}_{c=1}^K$ are trained simultaneously, each learning to separate class c from all others; the predicted class is $\arg \max_c (\mathbf{w}_c \cdot \mathbf{x})$. By the Perceptron Convergence Theorem, this update rule is guaranteed to converge *only if the classes are linearly separable*; otherwise, the algorithm oscillates indefinitely without ever finding a stable separating hyperplane.

3.2 Multilayer Perceptron (MLP)

An MLP stacks one or more hidden layers of the form $\mathbf{h}^{(l)} = \phi(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$ between the input and a softmax output layer, where ϕ is a non-linear activation (ReLU, Sigmoid/Logistic, or Tanh). Parameters are trained via **backpropagation** to minimize the multiclass cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^K y_{i,c} \log(\hat{y}_{i,c}) \quad (3)$$

using gradient-based optimizers such as plain Stochastic Gradient Descent (SGD) or Adam (adaptive per-parameter moment estimation). Unlike PLA, the composition of non-linear hidden layers allows the MLP to approximate arbitrary non-linear decision boundaries (Universal Approximation Theorem), at the cost of a substantially larger, non-convex hyperparameter and optimization search space.

4 Dataset Description & Preprocessing

Attribute	Specification
Dataset Name	English Handwritten Characters Dataset
Total Samples	3,410 images (exactly 55 per class, perfectly balanced)
Number of Classes	62 (digits 0–9, uppercase A–Z, lowercase a–z)
Raw Image Format	1200×900 RGB, black ink on white background
Preprocessing	Grayscale conversion → resize to 28×28 (Lanczos) → flatten to 784-d → normalize to [0, 1]
Label Encoding	<code>LabelEncoder</code> (62 classes → integers 0–61)
Train-Test Split	80% train (2,728 images, ~44/class) / 20% test (682 images, ~11/class), stratified, seed=42

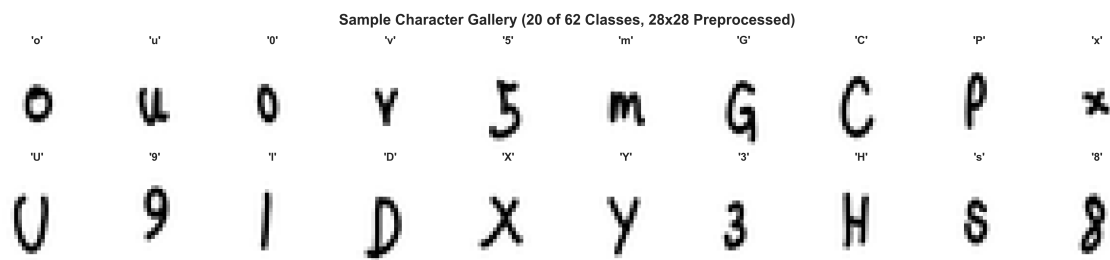


Figure 1: Sample Character Gallery — 20 of 62 Classes, Preprocessed to 28×28 Grayscale.

5 Model A: PLA Implementation & Results

5.1 Training Convergence

Table 2 and Figure 2 show PLA's train/test accuracy and misclassification count across 60 training epochs.

Table 2: PLA Training Convergence (Selected Epochs)

Epoch	Train Acc.	Test Acc.	Weight Updates
1	3.70%	3.81%	2,689
10	14.81%	11.29%	2,544
20	24.71%	14.96%	2,427
30	33.54%	19.50%	2,369
40	28.37%	15.40%	2,325
50	28.37%	17.60%	2,283
60 (Final)	37.72%	18.62%	2,253

Observation: PLA never converges to a stable solution — accuracy and misclassification counts oscillate rather than monotonically improving, because the 62-class handwritten-character feature space is not linearly separable.

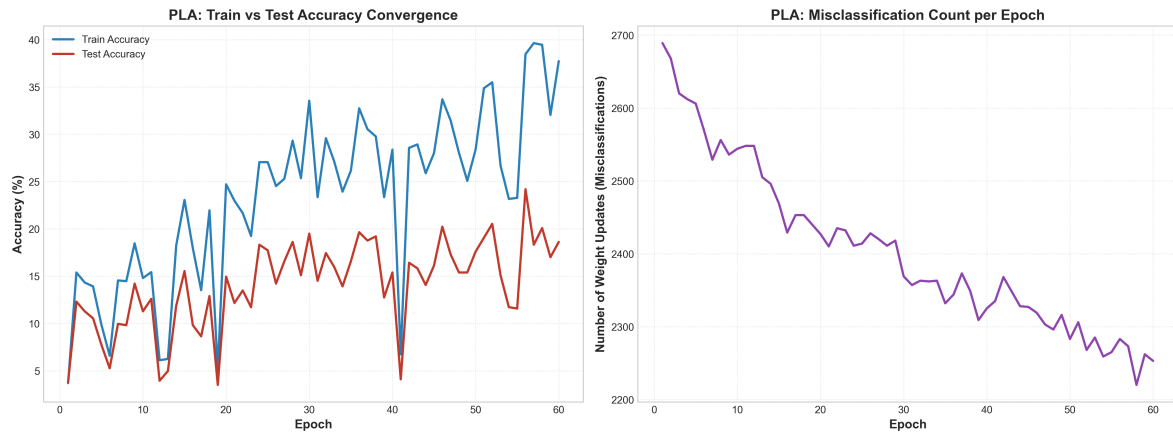


Figure 2: PLA Train vs Test Accuracy Convergence, and Misclassification Count per Epoch.

5.2 Learning-Rate Sensitivity

Table 3: PLA Learning-Rate (η) Sensitivity (20 Epochs)

Learning Rate (η)	Final Test Accuracy
0.001	14.96%
0.01	14.96%
0.1	14.96%
1.0	14.96%

Observation: Accuracy is exactly identical across all four learning rates — a direct empirical confirmation that, with zero-initialized weights, the perceptron’s sign-based decision rule is invariant to uniform positive rescaling of η .

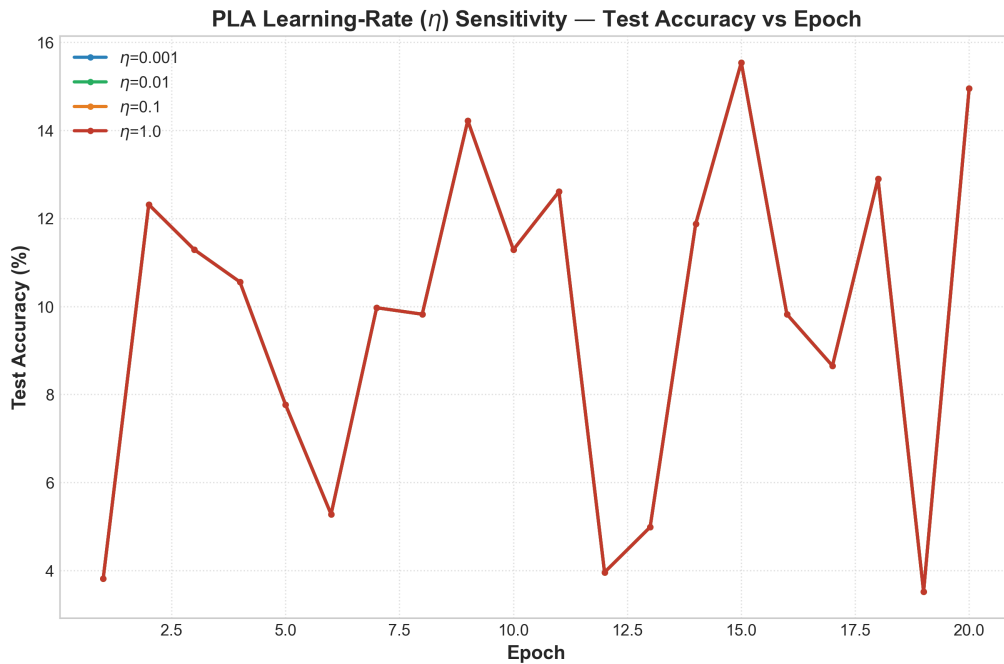


Figure 3: PLA Learning-Rate Sensitivity — Test Accuracy vs Epoch for $\eta \in \{0.001, 0.01, 0.1, 1.0\}$.

5.3 PLA Final Test Metrics

Table 4: PLA Final Test Set Performance

Metric	Value
Test Accuracy	18.62%
Precision (macro)	28.75%
Recall (macro)	18.62%
F1-Score (macro)	16.04%
ROC-AUC (micro-avg)	0.8000
ROC-AUC (macro-avg)	0.8510

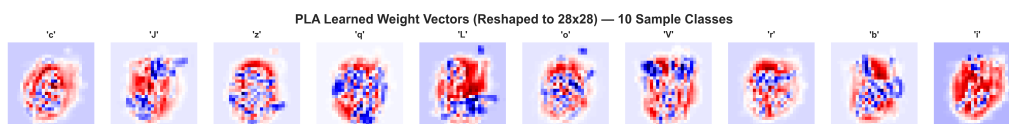


Figure 4: PLA Learned Weight Vectors, Reshaped to 28×28 , for 10 Sample Classes.

6 Model B: MLP Implementation & Hyperparameter Tuning

A staged hyperparameter search was used: (1) architecture $\times$ activation, (2) optimizer $\times$ learning rate (using the Stage 1 winner), (3) batch size (using the Stage 1+2 winner) — each evaluated with 3-fold Stratified Cross-Validation via `GridSearchCV`.

6.1 Stage 1: Architecture $\times$ Activation Function

Table 5: Stage 1 — Architecture x Activation (3-Fold CV Accuracy)

Hidden Layer Sizes	ReLU	Tanh	Logistic
(64,)	1.54% (dead)	16.97%	39.04%
(128,)	1.54% (dead)	31.01%	40.18% (Best)
(128, 64)	19.24% (unstable)	26.10%	39.19%
(256, 128, 64)	1.54% (dead)	16.42%	31.27%

Observation: ReLU catastrophically fails (accuracy $\approx$ chance level, 1.5%) across almost every architecture — a classic **dying ReLU** failure mode, caused by normalized $[0, 1]$ pixel inputs with a large zero-valued background, small default weight initialization, and a limited 150-iteration training budget, which together drive a large fraction of ReLU units to permanently saturate at zero. **Logistic (Sigmoid)** activation was the clear winner at every architecture size, and the shallowest single-hidden-layer network **(128,)** outperformed all deeper architectures.

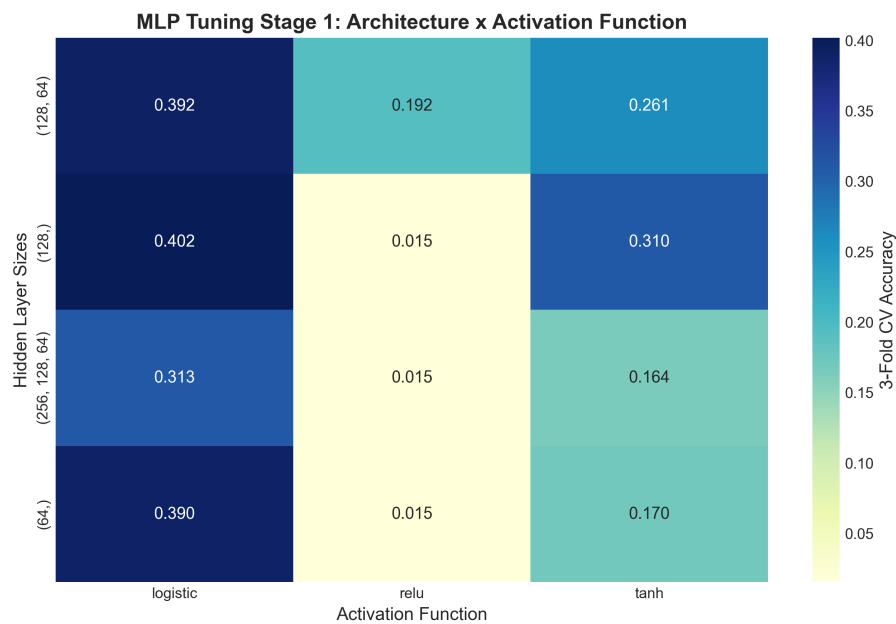


Figure 5: MLP Tuning Stage 1: Architecture x Activation Function Grid Search Heatmap.

6.2 Stage 2: Optimizer $\times$ Learning Rate

Table 6: Stage 2 — Optimizer x Learning Rate (Best Architecture: (128,), Logistic)

Optimizer	Learning Rate	3-Fold CV Accuracy
SGD	0.0001	3.48%
Adam	0.0001	28.12%
SGD	0.001	16.64%
Adam	0.001	40.18% (Best)
SGD	0.01	40.14%
Adam	0.01	1.58% (diverged)
SGD	0.1	22.98%
Adam	0.1	1.58% (diverged)

Observation: Adam (lr=0.001) and SGD (lr=0.01) reach nearly identical peak accuracy ($\sim 40\%$) through opposite sensitivity profiles — Adam converges well only within a narrow, small learning-rate band and **diverges catastrophically** at $\text{lr} \geq 0.01$, while SGD needs a larger learning rate to make progress within the fixed iteration budget but degrades far more gracefully as lr increases further.



Figure 6: Left: Optimizer x Learning-Rate Grid Search. Right: SGD vs Adam Convergence Speed (Same Architecture).

6.3 Stage 3: Batch Size

Table 7: Stage 3 — Batch Size (Best Config: (128,), Logistic, Adam, lr=0.001)

Batch Size	3-Fold CV Accuracy
16	37.50%
32	40.18%
64	40.87% (Best)
128	39.63%

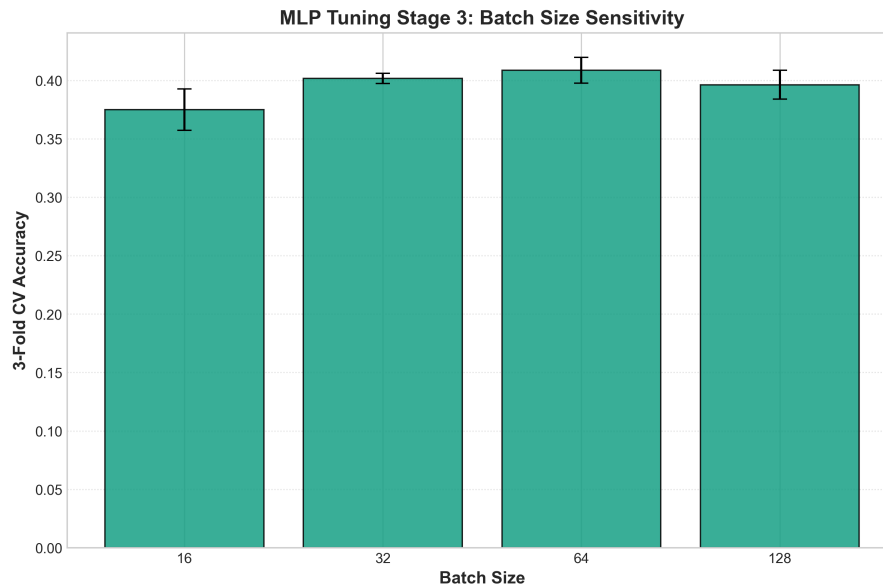


Figure 7: MLP Tuning Stage 3: Batch Size Sensitivity.

6.4 Final Chosen MLP Configuration

Table 8: Final MLP Hyperparameters and Justification

Hyperparameter	Selected Value	Justification
Architecture	(128,) — single hidden layer	Outperformed deeper nets; small ~ 44 samples/class cannot support more parameters
Activation	Logistic (Sigmoid)	Only activation avoiding the dying-unit failure on this input distribution
Cost Function	Cross-Entropy (log-loss)	Standard for softmax multiclass classification
Optimizer	Adam	Best CV accuracy at its optimal learning rate
Learning Rate	0.001	Best Adam configuration; avoided divergence seen at $\text{lr} \geq 0.01$
Batch Size	64	Best 3-fold CV accuracy; balances gradient noise vs convergence speed

6.5 MLP Final Test Metrics

Table 9: MLP Final Test Set Performance

Metric	Value
Test Accuracy	38.56%
Precision (macro)	42.02%
Recall (macro)	38.56%
F1-Score (macro)	37.93%
ROC-AUC (micro-avg)	0.9069
ROC-AUC (macro-avg)	0.9037

7 A/B Comparison: PLA vs Tuned MLP

Table 10: A/B Comparison Summary

Metric	PLA (Model A)	MLP – Tuned (Model B)	Improvement
Test Accuracy	18.62%	38.56%	+19.94 pp (2.07x)
Precision (macro)	28.75%	42.02%	+13.27 pp
Recall (macro)	18.62%	38.56%	+19.94 pp
F1-Score (macro)	16.04%	37.93%	+21.89 pp (2.36x)
ROC-AUC (micro-avg)	0.8000	0.9069	+0.1069
ROC-AUC (macro-avg)	0.8510	0.9037	+0.0527
Training Convergence	Oscillates, never stabilizes	Smooth, monotonic improvement	MLP converges; PLA does not

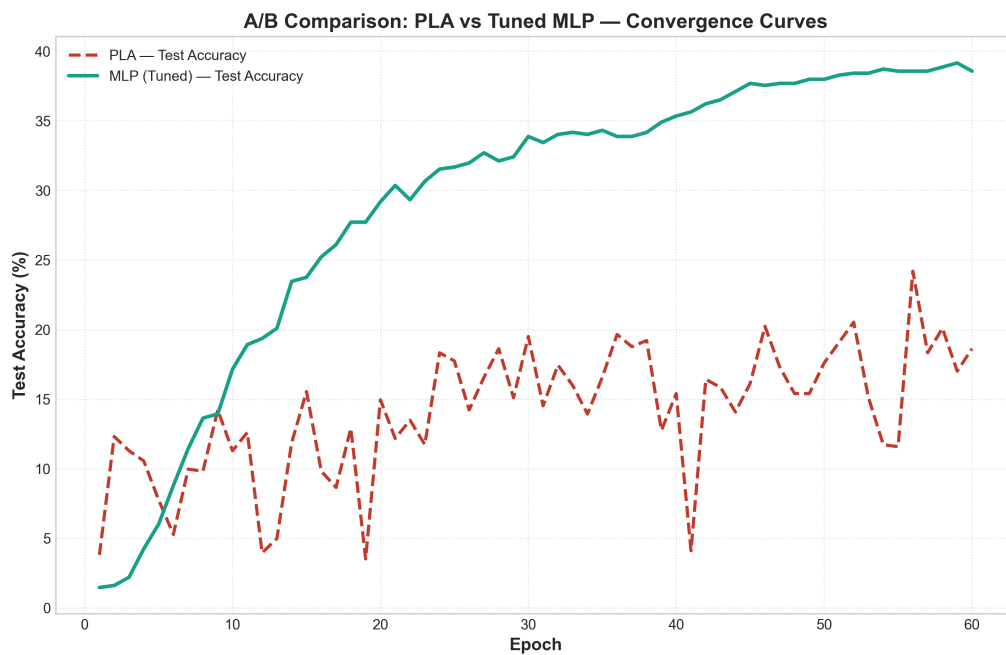


Figure 8: A/B Comparison: PLA vs Tuned MLP — Test Accuracy Convergence Curves.

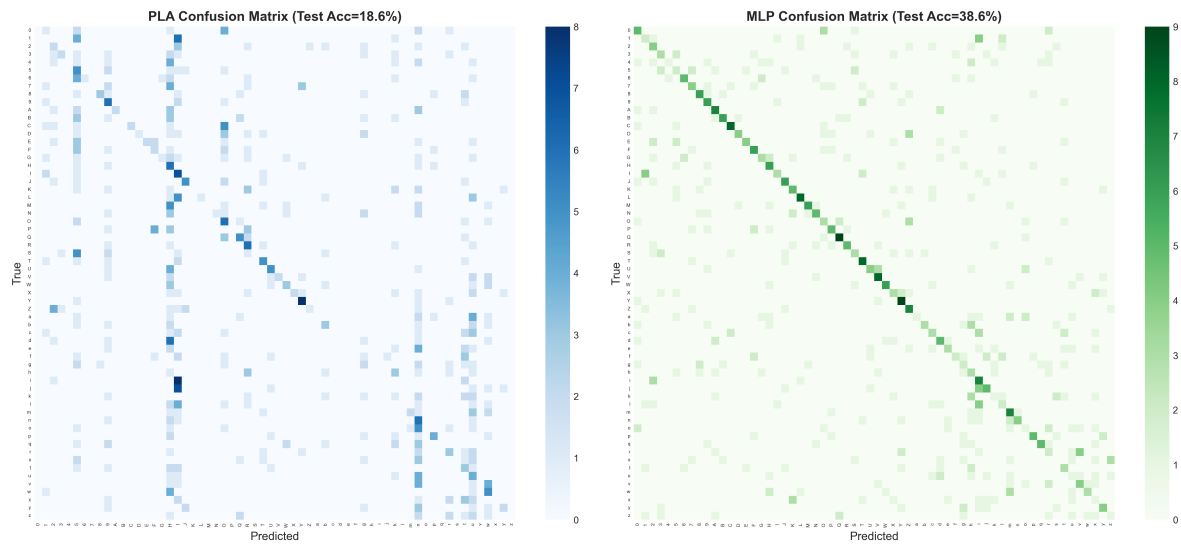


Figure 9: Confusion Matrices (62x62, Test Set): PLA (Left) vs MLP (Right).

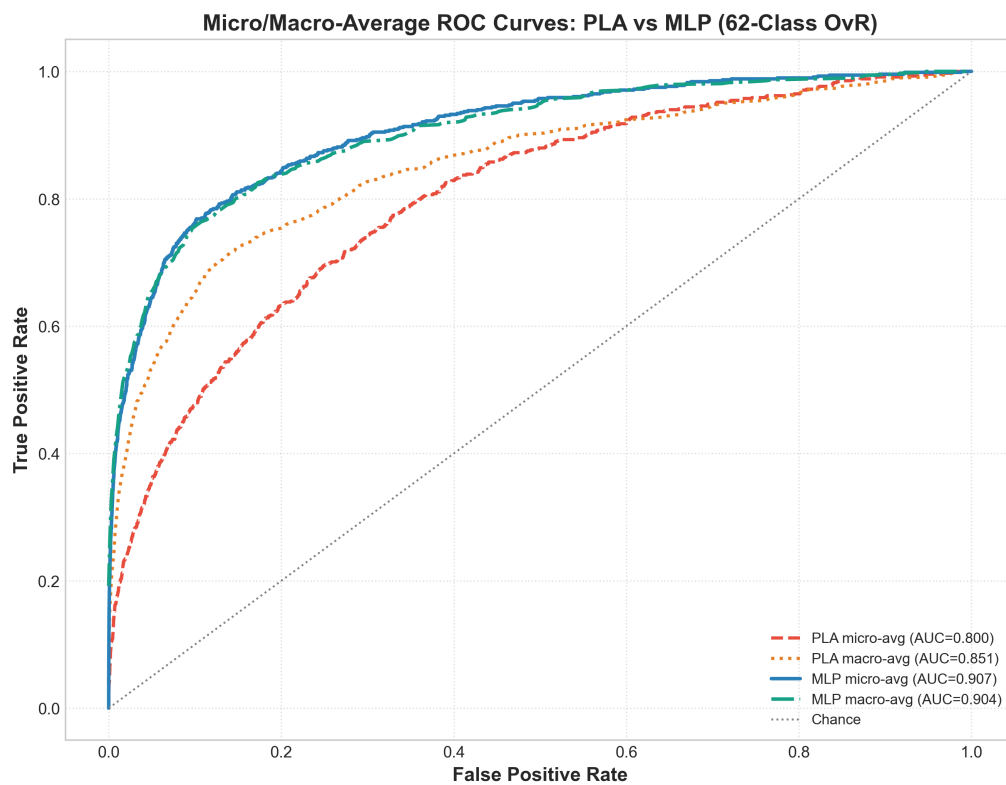


Figure 10: Micro/Macro-Average ROC Curves: PLA vs MLP (62-Class One-vs-Rest).

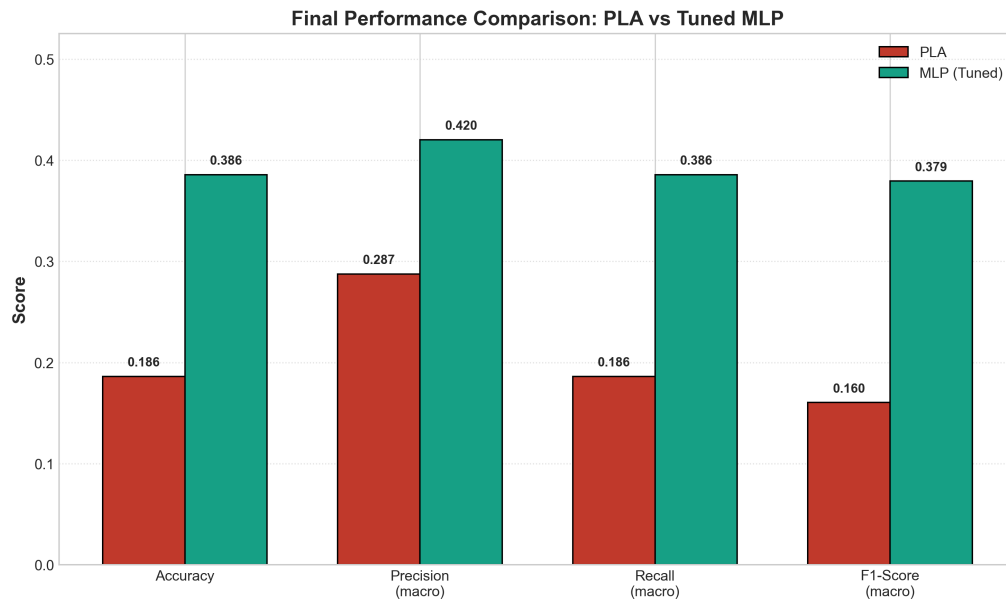


Figure 11: Final Performance Comparison: Accuracy, Precision, Recall, and F1-Score (macro) — PLA vs Tuned MLP.

8 Answers to Observation Questions & Detailed Discussion

8.1 1. Why does PLA underperform compared to MLP?

PLA is a strictly linear classifier — each of its 62 One-vs-Rest sub-perceptrons can only carve the 784-dimensional pixel space with a single hyperplane. Handwritten character classes are not linearly separable: character pairs such as '0'/'O' or '1'/'l'/'1' differ by subtle, non-linear stroke variations that no single hyperplane can cleanly separate from the other 61 classes simultaneously. This is directly visible in the training convergence curve (Figure 2): rather than settling, PLA's misclassification count and accuracy oscillate indefinitely across all 60 epochs, and its confusion matrix shows predictions collapsing onto a handful of "attractor" columns rather than a clean diagonal. MLP, by contrast, learns a non-linear decision boundary through its hidden layer, sigmoid activation, back-propagation, and the Adam optimizer — achieving a clean diagonal-dominant confusion matrix at roughly double PLA's accuracy.

8.2 2. Which hyperparameters had the most impact on MLP performance?

Activation function had the largest impact by far: switching ReLU → Logistic changed CV accuracy from ~1.5% (chance level) to ~40% at the same architecture. **Learning rate** was second, specifically its interaction with optimizer choice — Adam collapsed from 40.18% to 1.58% with just a 10x lr increase. **Architecture depth** had a clear but smaller effect: deeper networks consistently underperformed the single-layer (128,) network by 5–10 percentage points, since ~44 training images/class cannot support additional parameters. **Batch size** had the smallest impact, varying CV accuracy by only ~3.4 percentage points.

8.3 3. Did optimizer choice (SGD vs Adam) affect convergence?

Yes, substantially. At their respective optimal learning rates, SGD ($\text{lr}=0.01$, 40.14%) and Adam ($\text{lr}=0.001$, 40.18%) achieved nearly identical peak accuracy, but Adam required a 10x smaller learning rate and diverged catastrophically (to $\sim 1.6\%$, chance level) at $\text{lr} \geq 0.01$, because its adaptive per-parameter scaling amplifies effective step sizes once gradients shrink near convergence. SGD, lacking this amplification, degraded far more gracefully as learning rate increased (22.98% even at $\text{lr}=0.1$) — demonstrating that Adam’s fast, low-lr convergence advantage comes with a narrower stable learning-rate range than plain SGD.

8.4 4. Did adding more hidden layers always improve results? Why or why not?

No — deeper was consistently worse. With Logistic activation, CV accuracy fell monotonically with depth: $(128,) \rightarrow 40.18\%$, $(128,64) \rightarrow 39.19\%$, $(256,128,64) \rightarrow 31.27\%$; the same pattern held for Tanh. This reflects the classic bias-variance trade-off: with only ~ 44 training images per class, deeper/wider networks add far more trainable parameters than the data can support, causing them to overfit training folds and generalize worse. A single, moderately-sized hidden layer was the capacity sweet spot for this dataset’s scale.

8.5 5. Did MLP show overfitting? How could it be mitigated?

To a degree, yes — declining cross-validated (not just training) accuracy as network depth increases is itself an overfitting signature, and the final model’s test-accuracy convergence curve shows a narrowing but persistent train-test gap as epochs increase. This could be further mitigated by: (a) L2 weight regularization (`alpha` in `MLPClassifier`) to penalize large weights, (b) `early_stopping=True` with a held-out validation split, (c) data augmentation (small rotations/translations/elastic distortions) to synthetically enlarge the ~ 44 -sample-per-class training set, and (d) dropout-style regularization if migrating to a deep-learning framework such as PyTorch or TensorFlow.

9 Conclusion

This experiment provides a clear, quantitatively grounded A/B comparison between a linear (PLA) and non-linear (MLP) classifier on a genuinely hard, small-sample, 62-class handwritten-character recognition task:

1. PLA’s step-activation, hyperplane-only decision boundaries cannot separate the character classes, and its training never converges (accuracy oscillating around 15–20% for the entire 60-epoch run).
2. Systematic staged hyperparameter tuning — architecture/activation, then optimizer/learning-rate, then batch size — allowed the MLP to more than double PLA’s test accuracy (**18.62% $\rightarrow$ 38.56%**) and macro-F1 score (**16.04% $\rightarrow$ 37.93%**), with a substantially higher and smoothly-converging ROC-AUC (0.80 $\rightarrow$ 0.91 micro-average).
3. The tuning process surfaced two important, non-obvious findings: **ReLU activation catastrophically fails** (“dies”) on this small, sparse, $[0, 1]$ -normalized dataset

while Logistic activation thrives, and **deeper architectures consistently underperform** a single hidden layer given the limited ~ 44 -samples-per-class training budget.

4. Both findings underscore that matching model capacity and activation choice to dataset scale and structure is as important as the headline choice between a linear and non-linear model.